

Railway Economy

Plan projektu portfolio — gra ekonomiczna, symulacja, analiza danych i AI

Cel: stworzyć grywalną symulację gospodarki kolejowej inspirowaną 18XX, która jednocześnie pokaże umiejętności programistyczne, modelowanie danych, testowanie, analizę danych oraz później uczenie ze wzmocnieniem.

Model współpracy: użytkownik pisze kod i buduje mechaniki. Asystent jest mentorem, project managerem i technical leadem: pilnuje kierunku, zakresu i architektury oraz robi review, ale nie wyręcza w implementacji.

1. Wizja i założenia

- Stała mapa USA z okresu rozwoju kolei, oparta na rzeczywistych relacjach przestrzennych i ważnych miastach.
- Spółki kolejowe kupują pociągi, projektują szlaki handlowe i przewożą towary między lokalnymi rynkami.
- Towary: żywność, materiały budowlane, surowce i dobra przemysłowe; dokładna lista po researchu historycznym.
- Miasta mają specjalizacje, lokalną produkcję, konsumpcję i rynki.
- Miasta są częściowo samowystarczalne: brak zainteresowania graczy może prowadzić do stagnacji, ale nie powinien automatycznie prowadzić do upadku.
- Transport może napędzać rozwój miasta, tworząc sprzężenia między handlem, produkcją i popytem.
- Wygrywa gracz o największym majątku, uwzględniającym aktualną wartość posiadanych akcji.

2. Technologie

Obszar	Technologia	Etap
Logika	czysty Python	od początku
Testy	Pytest	od początku
Dane	DataFrame/CSV → SQL	iteracyjnie
Analiza	Jupyter + Matplotlib/Seaborn	po symulacji
UI	Streamlit	później
Grafy	Dijkstra	wczesny etap
RL	Gymnasium + PyTorch	późny etap
Eksperymenty	TensorBoard / W&B	etap AI
Wersjonowanie	Git + GitHub	od początku

3. Spółki, akcje, pociągi i infrastruktura

- Docelowo około 10 spółek; prototyp może zaczynać od jednej spółki na gracza.
- Spółka ma własne finanse. Roboczo maksymalna pula akcji to 1000; emisja/skup może zmieniać liczbę akcji w obiegu.
- Prezesem jest inwestor z największym udziałem; kontrola może przechodzić między graczem i NPC.
- Spółka może wypłacać dywidendy lub zatrzymywać gotówkę na rozwój.
- Pociągi mają konfigurację lokomotywy i wagonów oraz ograniczoną pojemność.
- Prezes projektuje trasę handlową: miasto A → zakup towaru → miasto B → sprzedaż; możliwe pętle i ładunki powrotne.
- Tory są stałą infrastrukturą grafową. Awarie mogą wymuszać alternatywną, dłuższą trasę.

- Kilka spółek może korzystać z tej samej infrastruktury.

4. Model rynku i miast

- Cena nie jest prostym 'popyt – podaż'. Punktem wyjścia jest relacja zapasu do średniego zużycia, czyli Days of Supply.
- Duży zapas względem zużycia wywiera presję na spadek ceny, niski zapas — na wzrost.
- Rynek powinien reagować z pamięcią/wygładzeniem, aby pojedyncza dostawa nie powodowała absurdalnych skoków.
- Sezonowość i wydarzenia losowe mogą zmieniać popyt.
- Fabryki mają reagować na opłacalność produkcji: ograniczać niechciane dobra i zwiększać produkcję dóbr bardziej rentownych.
- Zmiany produkcji wpływają na popyt na surowce, tworząc łańcuchowe efekty w gospodarce.

5. Przebieg jednej tury

- Faza decyzji: wysyłanie/wstrzymywanie/edycja pociągów, akcje, lokomotywy i wagony.
- Commit: po zakończeniu tury decyzje gracza zostają zatwierdzone i rozpoczyna się symulacja.
- Pociągi: sprawdzenie trasy, ewentualny objazd, depozyt kosztów, ruch o dzienny dystans.
- Miasta: konsumpcja, postęp produkcji, ukończenie produkcji.
- Rynki: aktualizacja zapasów i cen.
- NPC: decyzje inwestorów po aktualizacji sytuacji gospodarczej.
- Koniec dnia: agregacja i zapis danych oraz przejście do kolejnego dnia.

6. Logowanie danych

- Events: TRAIN_DISPATCHED, TRAIN_ARRIVED, CARGO_SOLD, CITY_CONSUMPTION, PRODUCTION_COMPLETED, MARKET_UPDATED, STOCK_BOUGHT/ SOLD, TRACK_FAILURE itd.
- State: gotówka, zapasy, pozycje pociągów, własność akcji, ceny, postęp produkcji i rozwój miast.
- Derived metrics: Days of Supply, zysk, ROI, estymowany zysk, błąd prognozy, rentowność trasy itd.
- Dla podróży przechowujemy planned vs actual: dystans, czas, koszt, przychód i wynik.
- Przykładowe dane podróży: pociąg/skład, A/B, towar, ilość, cena zakupu, estymowany przychód, depozyt, koszt rzeczywisty, cena sprzedaży, objazd/opóźnienie.

7. Kluczowe zasady architektoniczne

- Rozdzielamy dane względnie stałe, zmienny stan, decyzje, zdarzenia i wartości pochodne.
- Nie przechowujemy osobno wartości, które można wiarygodnie wyliczyć z innego stanu — np. prezes może wynikać z aktualnego akcjonariatu.
- Najpierw prosty działający model, potem testy, dane i kolejne warstwy. Nie budujemy architektury na zapas.
- Repository Pattern, SQL, MDP i RL wchodzi dopiero wtedy, gdy prosty silnik domenowy będzie działał.

8. Roadmapa

Etap	Cel	Rezultat
0. Fundament	Git, domena, graf, testy	mały działający silnik
1. Core simulation	dzień, pociągi, miasta	prototyp tekstowy
2. Data layer	eventy + CSV/DataFrame	pierwsze zbiory danych
3. Market	popyt, podaź, Days of Supply, produkcja	działający model rynku
4. Stocks/NPC	giełda, inwestorzy, dywidendy	żywy rynek kapitałowy
5. Analysis	Jupyter, EDA, wizualizacje	case study do portfolio
6. Streamlit	interfejs	grywalna aplikacja
7. SQL/Repository	oddzielenie domeny od persystencji	migracja CSV → SQL
8. RL	Gymnasium + MDP + PyTorch	boty uczące się gry
9. Experiments	TensorBoard/W&B	porównania agentów
10. Portfolio	README, dokumentacja, demo	publiczny projekt

9. Offline Sprint 0 — praca bez internetu

Cel: wrócić z małym fundamentem, który rozumiesz. Nie próbuj budować połowy gry.

- Skonfiguruj projekt Python i Git; naucz się podstaw: init, status, add, commit, log, branch, switch.
- Utwórz minimalną strukturę src/tests.
- Zaimplementuj minimalne: City, Goods, Company, Train, Player.
- Zaimplementuj RailwayGraph i prostą wersję Dijkstry.
- Zaimplementuj pozycję i dzienny ruch pociągu.
- Zaimplementuj minimalny Game/GameState wykonujący jeden dzień.
- Dodaj Pytest: obiekty, poprawna trasa, ruch, limit dystansu, zmiana dnia.
- Rób małe, opisowe commity.
- Prowadź DECISIONS.md z decyzjami i pytaniami otwartymi.

Proponowana struktura Sprint 0

```
railway-economy/  
  ■■■ src/railway_game/  
    ■ ■■■ domain/  
      ■ ■■■ city.py  
      ■ ■■■ company.py  
      ■ ■■■ train.py  
      ■ ■■■ goods.py  
      ■ ■■■ player.py  
      ■ ■■■ simulation/game.py  
      ■ ■■■ infrastructure/railway.py  
    ■■■ tests/  
    ■■■ README.md  
    ■■■ DECISIONS.md  
    ■■■ .gitignore  
    ■■■ pyproject.toml
```

10. Czego nie robić w Offline Sprint 0

- Nie implementować SQL, Streamlit, PyTorch ani Gymnasium.
- Nie robić pełnego rynku, NPC, dywidend ani skomplikowanej księgowości.
- Nie robić historycznie kompletnej listy miast i towarów.
- Nie budować dużej architektury 'na zapas'.
- Nie kopiować rozwiązań bez zrozumienia ich działania.

11. Definition of Done

- ✓ repozytorium Git i sensowne commity
- ✓ README i DECISIONS.md
- ✓ minimalne modele domenowe
- ✓ RailwayGraph + Dijkstra
- ✓ prosty ruch pociągu
- ✓ minimalny Game/GameState
- ✓ kilka wartościowych testów Pytest
- ✓ kod, który potrafisz samodzielnie wyjaśnić

12. Otwarte decyzje — na później

- Dokładna formuła cen i wygładzania rynku.
- Sprzedaż towaru po dotarciu: natychmiast, decyzja gracza czy wcześniej wydane zlecenie.
- Dokładny model produkcji fabryk i zmiany profilu produkcyjnego.
- Podatki i ich historyczna poprawność.
- Dywidendy i ich wpływ na akcje.
- Wartość fundamentalna spółki.
- Dokładne zachowanie NPC i późniejsze MDP.
- Repository Pattern i migracja CSV → SQL.
- Historyczny research USA, miast, towarów i kolei.

13. Główna zasada

Najpierw prosty działający model → testy → dane → analiza → interfejs → AI.

Projekt ma rosnać razem z Twoimi kompetencjami. Nie liczy się ilość kodu, lecz to, czy rozumiesz system, potrafisz go testować, analizować i rozwijać.